System, User, and Assistant Messages

Message roles

System messages carry application instructions and boundaries. User messages contain the current request. Assistant messages can be included to preserve conversation context or demonstrate examples.

The provider doesn't automatically know previous API calls unless the application sends prior context again.

Prompt placement

Stable rules usually belong in the system message or configuration. User-specific content belongs in the user message.

Few-Shot Prompting and JSON Output

Few-shot examples

Examples are useful when the model must follow a pattern, especially for classification, extraction, or transformations with borderline cases.

JSON

When downstream code parses model output, define expected fields clearly. Validate the result before assuming every key is present or every value has the right type.

Temperature, Tokens, and Streaming

Temperature

Lower temperature generally reduces variation. It doesn't guarantee truth or identical output across every provider/model configuration.

Tokens

Messages, retrieved context, tool messages, and generated text all consume context. Context windows are finite.

Streaming

Streaming returns partial output as it is generated. Non-streaming waits for the full response.

OpenRouter and Groq Request Notes

Request shape

Both can be used with chat-style message arrays. Provider settings, model names, base URLs, and authentication differ even when the overall structure is similar.

Errors

Handle invalid keys, rate limits, timeouts, malformed responses, and provider errors. Don't print secrets in logs.

Embeddings Basics

Meaning

An embedding is a fixed-length numerical vector representing text. Similar meaning often produces vectors that are closer under a similarity measure.

Model consistency

Documents and queries should normally be embedded with the same model. all-MiniLM-L6-v2 produces 384-dimensional vectors.

Cosine Similarity and Semantic Search

Cosine similarity

Cosine similarity measures how aligned two vectors are. A higher score often means more semantic similarity, but the score isn't a universal probability.

Semantic vs exact

Semantic search can match paraphrases. Exact identifiers and error codes may still benefit from keyword or filtered search.

Chunking Strategies

Fixed-size chunks

Fixed-size chunking is simple and predictable. Overlap can preserve context across boundaries, but too much overlap creates redundancy.

Natural boundaries

Paragraph or section boundaries can keep related sentences together. Smaller isn't always better, and larger isn't always better.

Chroma and FAISS Notes

Chroma

Chroma stores vectors with metadata and supports local collections that are convenient for small projects.

FAISS

FAISS focuses on efficient similarity search. Applications often keep metadata separately or wrap FAISS in another layer.

Top-k and Retrieval Quality

Top-k

A small k may miss supporting context. A large k may add noise and use more prompt space. Test the behavior rather than assuming more is better.

Debugging

Inspect retrieved chunks and scores before blaming the generator.

RAG Prompting and Grounding

Flow

Question, retrieval query, retrieved chunks, prompt context, model answer. Each stage can fail separately.

Unsupported questions

If the application promises source-based answers, define what happens when the documents don't support the answer.

RAG Failure Cases

Retrieval failure

The correct passage never appears in retrieved context. Check chunking, query wording, embedding model, metadata filters, and k.

Generation failure

The correct passage is present, but the answer ignores or contradicts it. Check instructions and output handling.

Practical RAG Handbook

From document to searchable chunks

Start by extracting readable text and keeping basic metadata such as file name, page, section, or chunk id. Metadata makes later debugging and source display much easier.

Chunking is a tradeoff. If a rule and its exception land in different chunks, retrieval may return only half the meaning. If chunks are huge, the retrieved context may contain several unrelated topics.

Overlap can protect content near boundaries. It also duplicates text, so use enough to preserve nearby context without making every chunk almost the same.

Embeddings

Embed stored chunks and incoming search queries with the same embedding model. The vector dimensions must be compatible.

Semantic search is useful for paraphrases. It can still struggle with exact identifiers, codes, or rare names, so metadata filters or keyword checks may help in those cases.

Retrieval

Top-k determines how many results you take forward. More results can improve coverage, but can also dilute the prompt with weak matches.

Always inspect what the retriever returned. If the correct passage is absent, the generation step cannot reliably recover the missing evidence.

Prompt construction

Put retrieved context in a clearly marked section and tell the model how it should use it. If the application is source-bound, say what to do when the answer isn't supported.

Don't silently mix old conversation, unrelated chunks, and instructions into one giant block without labels.

Debugging

Record the original question, rewritten search query, retrieved chunk ids, scores, final context, and answer. This gives you enough information to separate retrieval mistakes from answer-generation mistakes.

Test direct questions, paraphrases, vague follow-ups, questions requiring two nearby facts, and unsupported questions.

Context Windows and Raw History

Growing history

Sending every message forever increases token use and can exceed the context window.

RAG issue

A vague follow-up may make sense to the model when full history is included but still be a poor search query for the vector store.

Buffer and Summary Memory

Buffer

A buffer keeps recent messages exactly and drops older ones. It is simple and useful for short conversations.

Summary

A summary compresses older history. It can keep the gist while losing exact details, so critical identifiers or numbers may need separate state.

Query Rewriting

Standalone queries

A follow-up such as 'what about the second one?' can be rewritten using recent history into a standalone query that the retriever can understand.

Topic changes

A good rewriter should leave a genuinely new topic alone instead of forcing old context into it.

Function Calling

Tool definitions

The model receives tool names, descriptions, and argument schemas. It can ask to call a tool, but the application executes the real function.

Validation

Validate tool arguments before execution. Treat model-generated arguments as untrusted input.

Tool Loops and Iteration Limits

Loop

A tool-using assistant may call a tool, receive the result, then decide whether another tool is needed.

Limits

Set a maximum number of tool steps. Repeatedly calling the same tool with the same input is usually a sign the loop is stuck.

Router Patterns

Routes

A router can choose between document search, tool use, and direct conversation. Routes should match the user's intent, not superficial details such as whether a message contains a number.

Ambiguity

If the route is unclear and the difference matters, the system can ask for clarification.

Conversational Agent Handbook

Conversation state

The model doesn't retain API calls by itself. The application stores messages, summaries, or structured state and includes what is needed in later requests.

A recent-message buffer is simple. A summary helps with long conversations but may drop exact details. Important IDs or selected options can be stored separately when needed.

Query rewriting

A follow-up such as 'what about the second one?' is meaningful in conversation but weak for vector search. Rewrite it into a standalone query using recent context.

A new topic should remain a new topic. Don't force old context into every rewrite.

Tool use

Tool descriptions should make it clear when each tool applies. The model chooses a tool and arguments, then application code validates and executes the real function.

Useful logs show tool name, arguments, result, and step number. They don't need to expose hidden reasoning.

Multi-step work

Some questions need retrieval followed by another action. For example, the assistant may find a monthly fee in a handbook and then call a calculator.

Set a maximum number of steps. If the model repeats the same search with the same input, stop and handle the loop instead of letting it run forever.

Routing

A router can send a greeting to a direct path, a policy question to retrieval, and a calculation request to a tool path. Route by intent, not just keywords.

Ambiguous requests can be clarified when choosing the wrong path would materially change the result.

REST API Basics

Methods

GET reads, POST submits or creates, PATCH updates part of a resource, and DELETE removes.

Status codes

Use status codes that match the outcome. Missing resources should not normally produce 500.

FastAPI and Pydantic

FastAPI

Routes map HTTP requests to Python functions. Pydantic models can validate incoming JSON before endpoint logic runs.

Validation

Missing required fields or wrong types should produce controlled validation errors.

SQLite and Relational Basics

Tables

Separate related entities into tables when it keeps the data understandable. Keys connect related rows.

Queries

JOIN combines related rows. GROUP BY supports grouped counts and totals.

Session State and Persistence

Isolation

Separate conversation state by session ID. A global list can mix users.

Persistence

In-memory dictionaries disappear when the process restarts. SQLite can preserve messages and notes across restarts.

Git Branches, Commits, and Pull Requests

Branches

Use a feature branch for related work. Keep commits focused enough that the history is understandable.

PRs

A pull request should explain what changed, how to test it, and any known limitation.

Gitignore and Secrets

Secrets

API keys belong in environment variables or .env files that aren't committed.

History

Adding a file to .gitignore doesn't remove it from earlier commits. Rotate a secret if it was exposed.

FastAPI and SQLite Project Handbook

Request models

Define request models for chat, notes, incidents, or project-specific operations. Required fields should be obvious, and optional fields should have sensible defaults.

Validation errors should be returned cleanly. Don't allow malformed input to fall through and create confusing failures later.

Chat sessions

A POST /chat endpoint commonly accepts session_id and message. Load the relevant history for that session before running the pipeline.

Do not keep one global history for every user. Session isolation and persistence are separate concerns.

SQLite

SQLite can store sessions, messages, saved notes, review results, incidents, and other small project state. Use a schema that matches what the app needs rather than creating many unused tables.

Parameterized SQL should be used for values instead of concatenating raw user input into SQL strings.

API behavior

Return 404 when a requested saved item doesn't exist. Use 422 or validation errors for malformed requests. Reserve 500-series errors for server-side failures.

Add a simple health endpoint so you can confirm the server is running independently of the LLM provider.

Testing

Test the API with curl, Postman, or Swagger. Include multi-turn requests using the same session id and a second session to check isolation.

Restart the server at least once if persistence matters. An in-memory solution may look correct until the process restarts.

n8n Webhooks

Webhook

A webhook URL starts a workflow when it receives an HTTP request. It can connect a custom backend to a visual automation.

Use

For a small capstone, n8n is optional unless it supports a real flow such as logging or notification.

GitHub Actions Basics

Workflow

A YAML file can run checks on push. Typical steps are checkout, install dependencies, and run tests or linting.

Failure

A useful CI workflow must fail when code or tests fail. A permanently green workflow proves very little.

Tests and Linting Notes

Tests

Test deterministic parts directly. For LLM flows, use a small set of expected behaviors and mock external calls where practical.

Linting

A linter catches style and some code-quality issues automatically. It doesn't replace functional tests.

RAG Debug Trace Examples

Trace A

Original query: 'what about the second one?'. Rewritten query: 'What are the requirements for the sports scholarship?'. Retrieved chunks include the sports scholarship section.

Trace B

Original query: 'Can I freeze in first semester?'. Retrieved chunks discuss withdrawal and medical leave but not first-semester freezing. A confident yes would be unsupported.

Tool Call Debug Examples

Repeated search

The assistant calls document_search four times with the same query even though the needed fee was already returned. The next missing step is a calculation.

Bad argument

The model sends top_k='three' when the tool expects an integer. Validate or repair before execution.

API Error Examples

Invalid key

Return a clear configuration or authentication error without printing the key.

Timeout

Catch the timeout and return a controlled error or retry according to the project's simple policy.

SQL Query Examples

JOIN

SELECT users.name, COUNT(orders.id) FROM users JOIN orders ON users.id = orders.user_id GROUP BY users.id; returns each matched user with an order count.

Wrong join

A syntactically valid JOIN can still connect the wrong columns and return believable but incorrect results.